AI ENGINEER (TAX) CASE STUDY · MIRAFLORES AI

Taxmark

A web-native tax workpaper where every number on the return carries its receipt.

Aarvin George · 17 October 2026

Prototype	taxmark-workpaper.vercel.app
Video	[VIDEO LINK]

TRY IT IN TWO MINUTES

1. Open the link. No sign-in: **Viewing as** (top right) switches between the preparer, reviewer, partner, clients and others.
2. As the reviewer, open **Checks** and click any amount: the right panel shows the source page with the field highlighted, the rule that built it, who checked it and what is open.
3. To start clean, add a workspace name to the link (for example `/w/your-name/daniel`) or click **Reset to start**.

CONTENTS

1. Approach and key decisions
2. The core idea and the data model
3. The errors found, and how the design surfaced them
4. How AI was used

1
Approach and key decisions

Live app: https://taxmark-workpaper.vercel.app (no sign-in; a role switcher, top right, shows every person's view).

The approach

A workpaper exists to prove the return. So in Taxmark every number is a claim that carries its proof. Click any number and the right-hand panel answers four questions: where it came from (the source page and field, highlighted), what was done to it (the rule that built it, and any change with who, why and the tax effect), who checked it (sign-offs bound to the value they signed), and what is still open (findings, review notes, questions).

I worked in this order: hunt the packet's confident mistakes first, let them shape the data model, build a thin end-to-end slice and deploy it early, then widen it in rounds, redeploying and re-testing after each.

Key decisions and their trade-offs

1. The packet is read-only; everything people do is an event. Adjustments, notes, sign-offs, questions, answers, reveals and clock moves are appended to a log in Postgres and never edited or deleted. An undo or a reset is a new event on top. Each screen is computed from the packet plus the log, for one person at one point in the story. *Trade-off:* every page replays the workspace's events (16 to 29 ms for 300 events, measured across roles and screens). In return the audit trail is complete by construction.

2. Our own checks, written against the packet's format, not against this client. Extraction confidence cannot catch a confident mistake: a wrong value read at 0.99 still reads at 0.99. So 15 families of deterministic checks re-derive what should agree: the return against the IRS and California account records, documents against each other, this year against last, and fields the engine ignored, such as a payment's tax year. No check names a document id, a person or a line key, and each family was run on an altered copy of the packet to prove it fires and stays quiet when it should. The engine's own diagnostics are shown beside ours under their own label. *Trade-off:* five families depend on how free text is worded (repair or improvement, home office, health coverage, state wages, the penalty method). A document that phrases the fact differently would be missed. That tier is where AI helps (below).

3. Roles are enforced on the server. Ten people from the packet's firm and household, from preparer to IRS examiner. Self-review is refused; a tax effect over 1,000 needs the partner; Marcus, without sensitive-data clearance, sees the last 4 digits of an SSN and cannot reveal more; clients see only their own items, in plain words. Masking happens in the API, so a hidden number never reaches the browser, and every reveal is logged. *Trade-off:* a role switcher instead of sign-in, so a reviewer can see every view from one link.

4. Every sign-off says what was checked. A sign-off is bound to the value it signed and goes stale if the value changes. The server refuses a sign-off on any item without a note on what was checked. On Checks, a reviewer ticks rows (or a whole severity group) in the one table and signs off the selection; if a ticked row has no note, nothing is signed and the screen opens that row. One undo reverses the batch. *Trade-off:* signing many items takes a note per item. That is the point: one note pasted on ten items is not a review.

5. One story clock replays the season, and the past is view-only. The packet's timeline (documents arriving, the e-file rejection, acceptance, a letter from the IRS years later) plays out as the clock moves. A finding appears when its evidence arrives, and the return freezes at federal acceptance; later changes are amended on top. You can move the clock back to look, but you can only act at the latest point the story has reached, so the filed return cannot be rewritten through the past. *Trade-off:* one clock per workspace, so two people cannot sit at different moments of the story at once.

6. The finalized return is v3 plus our applied adjustments, not a recomputation. Each change shows its tax effect estimated from the packet's own rates, marked as an estimate. *Trade-off:* totals do not cascade; a real product would send the adjusted inputs back to the engine.

7. Simulated source pages, built to be replaced. Each page is rendered from the packet's printed text with every extracted field highlighted at its recorded position. Every trace already points to a document, page and position, so a real PDF can take the simulated page's place without changing anything upstream.

8. Client data stays out of the code. The packet and the AI's suggestions are stored encrypted (AES-256-GCM) in private cloud storage. The app opens the packet by its fingerprint (SHA-256) and refuses it if one byte has changed; the keys exist only on the server. *Trade-off:* one more moving part at deploy time, checked by reading the data back before every build goes live.

9. One place per action, one word per meaning. A late review of the whole app removed repeated screens and actions: the separate sign-off list became a checkbox column in the Checks table, Home stopped repeating the bell, each row has one link per destination, and the right panel names what its actions act on. The note that can block approval is a "review note"; the note on a sign-off is "what you checked".

Designed, not built

Uploading a real file as evidence (notes are built), the examination response pack (the examiner already sees the items under examination and the return as filed), the penalty and interest estimate, many returns behind sign-in (the storage is already keyed by each packet's fingerprint), and a cited source on every tax definition.

How it was tested

515 unit tests run in continuous integration on every push, with a typecheck, a scan for full SSNs and the AI scorecard gate; 79 browser tests run before every push; and a 36-step walk through the brief's requirements runs against the live site after each deploy (36 of 36 at the last deploy). Each known mistake has a test that fails if the app stops flagging it.

2

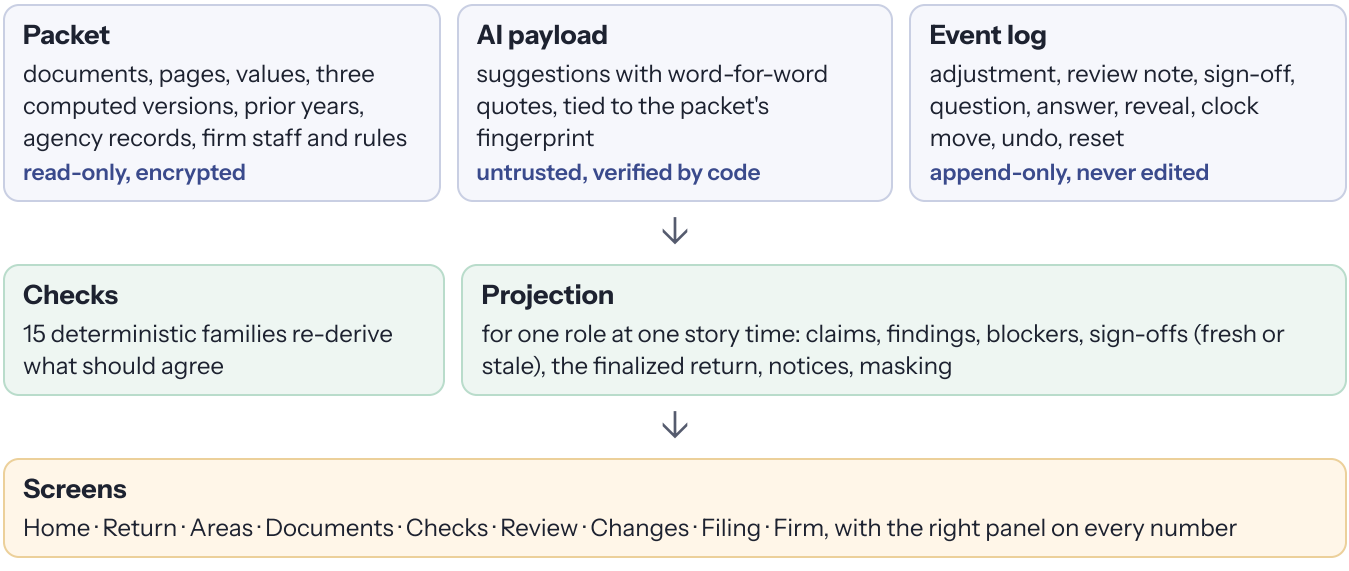
The core idea and the data model

The core idea

Every number is a claim that carries its proof. A workpaper exists to prove the return, so Taxmark makes each number answer four questions in one click: where it came from (the source page and field), what was done to it (the rule, and any change with who, why and the tax effect), who checked it (sign-offs bound to the value they signed), and what is still open (findings, review notes, questions). Nothing is a cell to be ticked; everything is evidence that can be followed, challenged and signed.

The data model

Three inputs, one log, and views computed per person.



- **Claim:** anything that can carry evidence and actions: a return line (`line:F540NR.72`), a document (`doc:D68`), a finding (`finding:…`) or the return as a whole.
- **Event:** who did what to which claim, when (wall time and story time), with a payload (new value and reason, note text, the signed value and a note on what was checked, method of verification). Never edited or deleted; an undo or a reset is a new event.
- **Finding:** produced by a check (or a verified AI suggestion) with its evidence references, severity, tax effect and the time it becomes visible in the story.
- **Role:** from the packet's firm staff and client users, with clearance and the screens it may open; the server enforces every action and masks identifiers before anything reaches the browser.
- **Story clock:** one per workspace; everything after it is hidden, and the return freezes at federal acceptance. The workspace also records the latest point it has reached: earlier points can be viewed but not written to.

Why this model

- **The audit trail is the data, not a feature.** Because state is the packet plus the log, "who changed what and why" and "what did the return look like on 13 July" are queries, not extra code.
- **Real documents can replace simulated ones.** Every trace points to a document, page and position; a PDF behind the same address changes nothing upstream.
- **Untrusted inputs are treated alike.** The engine's numbers, the extraction's confidence and the AI's suggestions all have to prove themselves against the packet before a reviewer relies on them.

3

The errors found, and how the design surfaced them

Generated from the app's own catalogue of checks and their output on the real packet, so every row here is what the app shows on its Findings page. Ids starting GT- are our ledger numbers for each mistake.

How they were found

Before building, we hunted the packet for confident mistakes: four AI researchers (Claude Opus 5.5), one per area, each reading the printed documents against the extracted values, the three computed versions, the IRS and California Franchise Tax Board (FTB) account records, the prior year and the client's own answers. Each finding was logged with its document, field, value, why it is wrong and its evidence; findings were spot-checked against the printed pages. Reviewers, including a second model (Claude Fable 5.1), raised and checked doubtful items, and the main session resolved them before the build. Two entries turned out to be the same mistake logged twice, so 29 ledger ids collapse into **27 distinct mistakes**.

Then each mistake became a **deterministic check** in the app, written against the packet's schema and never against a document id, and tested on an altered copy of the packet to prove it can fire and stay silent. The app does not remember the answers; it re-derives them.

The 27 mistakes

Tax effect is the estimated change in tax from correcting the mistake, from the packet's own `tax_effect_rates` (plus means more tax owed). It is an estimate; the engine has not recomputed the return.

Id	The mistake	Caught by	In the app	Tax effect of the fix
GT-A01	RSU sales at zero basis; the stock-plan supplement (D11) ignored, so wage income is taxed twice	Blank read used as zero	Blocker	−15,536 federal, −2,765 California
GT-A02	California wages include Halvorsen pay earned after the move to Texas	Residency and sourcing	Blocker	−5,685 California
GT-A03	California estimated payments include a payment designated tax year 2024	Tie-out to agency record; Field the rule ignores	Blocker	+3,500 California
GT-A04	Federal Q4 estimate from the client's guess (4,000); the IRS transcript shows 3,000	Tie-out to agency record	Check failed	+1,000 federal
GT-A05	Theo's SSN transposed in the organizer; caused the July e-file rejection	Identity vs documents and prior year	Blocker	no dollar effect

Id	The mistake	Caught by	In the app	Tax effect of the fix
GT-A06 (also GT-C03)	Self-employed health insurance deducted though Erik could join Ngozi's employer plan (the client's own questionnaire answer)	Client answer vs line	Blocker	+4,464 federal, +605 California
GT-A07	Little Sprouts tax ID misread at 0.97 confidence	Two sources disagree	Check failed	no dollar effect
GT-A08	Wrong provider name on Form 2441	Two sources disagree	Check failed	no dollar effect
GT-B01	Northlake sweep interest on two forms, counted twice	Value counted twice	Check failed	−640 federal, −127 California
GT-B02	Swedish interest converted at the 2024 exchange rate, not 2025	Field the rule ignores	Check failed	+43 federal, +9 California
GT-B03	Crypto lot C6 at zero basis after the client sent the real cost (in a message that arrived after the last computed version, v3)	Answer after the last computation	Blocker	−1,312 federal, −195 California
GT-B04	Meridian lot M18 (page 32) never read in either 1099; the cause of the engine's own mismatch warning	Blank read used as zero	Check failed	−21 federal, −2 California
GT-B05 (also GT-C09)	Prior preparer's open item (2023 Form 3520, foreign inheritance) dropped	Open item never closed	Check failed	no dollar effect
GT-C01	1099-K card money counted twice in Schedule C gross receipts	Value counted twice	Blocker	−4,402 federal, −384 California
GT-C02	Mileage-log page scanned twice: 11,800 miles vs 8,100	Value counted twice	Check failed	+919 federal, +80 California
GT-C04	Phone, internet and Spotify at 100% business; call notes say 70%, 50%, personal	Client answer vs line	Check failed	+541 federal, +47 California
GT-C05	11,000 heat-pump replacement deducted as a repair; an improvement to depreciate	Judgment needed	Blocker	+3,633 federal, +896 California
GT-C06	Section 179 not elected for the equipment; saves California tax at no federal cost	Cheaper election available	Blocker	−1,050 California
GT-C07	SEP contribution over the limit; the excess was carried and never flagged	Limit or cap	Judgment	no dollar effect
GT-C08	Home-office simplified method blended across two homes	Judgment needed	Judgment	+195 federal, +17 California
GT-C10	2024 return claimed a 300 sq ft office; the room measures 220	Prior year vs documents	Check failed	no dollar effect
GT-D01	Lonestar payoff interest counted twice (the 1098 already includes it)	Value counted twice	Check failed	+559 federal, +103 California

Id	The mistake	Caught by	In the app	Tax effect of the fix
GT-D02	Home property tax includes the seller's share credited at closing	Two sources disagree	Blocker	+1,462 federal, +332 California
GT-D03	Donated car at the client's Blue Book guess, not the 1098-C sale price	Two sources disagree	Check failed	+806 federal, +149 California
GT-D04	California Schedule S credit for Oregon tax not limited (allowed for a composite return, but capped)	Limit or cap	Check failed	+209 California
GT-D05	Underpayment penalty ignores excess social security tax as withholding	Field the rule ignores	Check failed	−147 federal
GT-D06	Schedule CA column D lines list incomplete inputs (trace gap; numbers right)	Trace gap	Note	no dollar effect

10 of the 27 are blockers: the partner cannot approve the return while any is open. Net estimated effect of fixing all 27: **−8,436 federal, −4,261 California** (each estimate taken alone; fixes interact).

How the design surfaced them

Five design choices do the work, in this order of importance:

1. **Checks that do not trust the engine.** Each check re-derives what should agree: the return against the IRS and FTB account records, two documents against each other, this year against last, a field the engine ignored (a payment's tax year, an exchange rate's year), a client's answer against the line it should change. A wrong value read at 0.99 confidence is still caught, because confidence is never the test.
2. **One inbox, blockers first.** Checks lists every finding by severity with its amount and tax effect; the engine's own diagnostics sit beside ours under their own label, never mixed in.
3. **Trace in one click.** Every finding links to the claim it sits on; the side panel shows the source page with the field highlighted, the rule that built the number, who checked it and what is open.
4. **The story clock.** A finding appears only once its evidence has arrived (a late K-1, a client message after the last computation), so the inbox at any date shows what a reviewer could know then.
5. **Findings stop the return.** A blocker or a blocking review note disables approval; an adjustment that clears it keeps the original value, the reason and the tax effect, and needs the partner when the effect is over 1,000.

Per mistake, as the Findings page states it:

- **GT-A01.** Blank broker basis read as 0.00 matched to the supplement lot with the same acquisition date; the unused supplement value is shown beside each lot.
- **GT-A02.** State wages rebuilt per employer from W-2 box 16, or the paystub split at the move date where one exists, against the residency dates.
- **GT-A03.** The payment's own tax-year field (read correctly by the pipeline) is checked, and the line is tied out to the FTB account by tax year.

- **GT-A04.** Estimated payments tied out to the estimated-payment lines on the IRS account transcript; the part resting only on a client answer is named.
- **GT-A05.** Every organizer SSN compared with every document that prints one for the same person (the 2024 return already had it); a mismatch blocks e-file. Last 4 only.
- **GT-A06.** The questionnaire answer on eligibility is linked to the line it gates, with last year's treatment beside it.
- **GT-A07.** Each identifier read is compared with the printed page; a transposition is shown whatever the confidence.
- **GT-A08.** A line naming a party with its tax ID is checked against the name the cited document prints.
- **GT-B01.** Two income lines with the same amount from documents that print the same account are flagged as one payment.
- **GT-B02.** Every foreign amount shows its rate and the rate's year; income must use the tax year's average.
- **GT-B03.** A message after the last computation that supplies a missing fact is shown as "answered, not applied" on the line, and it appears in the story when it arrives.
- **GT-B04.** A partial extraction used by the return lists its unread pages and what they print; the summary gap is traced to them.
- **GT-B05.** Each prior-preparer open item is carried in and checked for a resolving document; the one with none stays open, linked to the documents that mention the same fact.
- **GT-C01.** Gross receipts are reconciled to the books. A 1099-K amount the books' deposit detail already includes is a duplicate, not an unmatched receipt.
- **GT-C02.** Pages whose extracted values repeat another page are flagged before their totals are summed.
- **GT-C04.** Call notes with a business share are linked to the books lines they describe.
- **GT-C05.** An invoice inside a repairs line that reads as a replacement goes to a repair-or-improvement judgment, with the engine's own what-if as the effect.
- **GT-C06.** Engine what-ifs that lower tax with no cost anywhere are presented as elections to decide.
- **GT-C07.** The contribution document is set beside the deductible limit; the excess becomes a decision item.
- **GT-C08.** A simplified home office with two areas in a year with two homes asks for a method per home.
- **GT-C10.** Prior-year figures are compared with this year's source documents, saying which year looks wrong.
- **GT-D01.** A lender statement that says it is reported on the Form 1098 is linked to the 1098 as detail, never a second source.
- **GT-D02.** A closing-statement tax credit that feeds no line is set beside the property-tax line for the same address, with the effect of the cap on deductible state and local taxes (SALT).
- **GT-D03.** A line taken from a client's estimate is shown against the third-party form for the same item.
- **GT-D04.** A credit equal to the raw other-state tax times a day ratio is checked against the limit to California's tax on that income.

- **GT-D05.** The penalty is recomputed from the packet (proven by reproducing the engine's figure first) with excess social security counted as withholding.
- **GT-D06.** Every sum line is recomputed from its listed inputs; where they do not reach the value, the missing sibling line is named.

What the AI reader found on its own

After the build we ran an AI reader (Claude Sonnet 5.5) blind over the packet, with none of these mistakes or their titles in any prompt. Judged by a second model, it found 13 of the 27 for the same reason (14 counting a partial). It also found one we had missed, now logged as **GT-D07**: the 1,500 gift to St. Brigid rests on a handwritten receipt (D64) with no statement of goods or services, which IRC 170(f)(8) requires for gifts of 250 or more; our hunt had treated a phone confirmation of the amount as enough. If disallowed: about +390 federal and +72 California. Of its other suggestions outside our list, a second model judged 8 wrong and 3 repeats of others, which is why every AI suggestion waits for a person. An earlier run had made three confident mistakes from a 1099 that was later replaced; a verifier rule now holds those back.

4
How AI was used

Two different uses: AI inside the product, where it reads and a person decides, and AI as the team that built it, where the work was checked by running it, not by reading it.

AI in the product: one job, checked by code

Rules are good with numbers. They cannot read "we replaced the heat pump" and decide whether that is a repair or an improvement. That is the one job given to AI.

- **When it runs.** An AI reader (Claude Sonnet 5.5) reads the packet once, never while someone browses: no model is called when a page loads. The suggestions shown today come from the scored run below, stored encrypted at deploy. The same reader also runs as a protected cloud function on a daily schedule (proven on a dry run) and only re-reads when the packet or the prompt changes.
- **What it sees.** Each document's text, with names and identification numbers masked first.
- **What it must produce.** A fixed format: each suggestion quotes the document word for word, names the document and page, gives a short reason (asked for 25 words, refused over 30) and, optionally, a legal source from a vetted list.
- **How it is checked.** Code treats the output as untrusted. A suggestion is held back if its quote is not on the page it names, if the quote carries no content, if it rests only on a document that was later replaced, if it cites a source outside the vetted list, or if it contains anything shaped like an SSN.
- **What a person sees.** What passes appears in the inbox of findings, marked with a star (✦) and "needs review". It never blocks filing; a reviewer accepts or dismisses it.

How well it works, measured before trusting it. A harness ran the reader blind (no known answer in any prompt, verified by searching every request) and scored it against the 27 mistakes found by hand.

Measure	Result
Automatic match score	17 of 27
Judged by a second model, match by match (the figure we report)	**13 of 27** (14 counting a partial)
Quotes shown that were verified on their page	all of them
Its other suggestions, outside the 27	8 judged wrong, 3 repeats, 1 held back by the verifier; each waits for a person
Confident mistakes in an earlier run	3, quoting a 1099 that was later replaced; a verifier rule now holds these back
Real mistakes it found that we had missed	**1** (a 1,500 gift with no written acknowledgment, IRC 170(f)(8))
Cost of one full run	about $1.50

A test in continuous integration fails the build if a suggestion is ungrounded, if recall falls below the measured baseline, or if the prompt changes without a new scored run.

AI as the build team

- **Claude Code (Claude Opus 5.5)** planned, wrote the code and tests, and coordinated parallel sub-agents, test first: wherever a test could catch the bug, the fix started from a failing test.
- **Claude Fable 5.1, a different model, reviewed each major round**, mostly by running probes against the running app. Its findings changed the product: a sign-off refused by the server used to erase the reviewer's notes; the story clock let the filed return be rewritten by moving back in time; evidence links were merged too aggressively. Each became a test before it was fixed.
- **Research and the mistake hunt.** Four AI researchers read the packet area by area before the build; reviewers, including a second model, raised and checked doubtful items before the ledger was settled.
- **What I decided.** The core idea, the scope, what AI may and may not do in the product, and every trade-off. Each decision is logged one line at a time with its trade-off, and every use of AI is logged as it happened.